

2024년 한이음 ICT멘토링 프로젝트 결과보고서

프로젝트명

AI 기반의 소스 및 바이너리 시큐어 코딩 점검 도구

요 약 본

프로젝트 정보	
프로젝트명	AI 기반의 소스 및 바이너리 시큐어 코딩 점검 도구
주제 영역	☐ 생활 ☒ 업무 ☐ 공공/교통 ☐ 금융/핀테크 ☐ 의료 ☐ 교육 ☐ 유통/쇼핑 ☐ 엔터테인먼트
기술 분야	☒ SW·AI ☐ 방송·콘텐츠 ☐ 블록체인·융합 ☒ 디바이스 ☒ 차세대보안 ☐ 미래통신·전파
성과 목표	☐ 논문게재 및 포스터 발표 ☐ 앱등록 ☐ 프로그램등록 ☐ 특허 ☐ 기술이전 ☐ 실용화 ☒ 공모전(한이음 공모전) ☐ 기타()
프로젝트 소개	ARM, x86 아키텍처 기반 장비의 취약점을 점검하기 위해 머신러닝 알고리즘을 사용해 취약 코드와 안전한 코드를 학습하고, AI 기반으로 코드의 취약성을 탐지합니다. 점검 결과는 PDF 그리고 웹 형식으로 생성됩니다.
개발 배경 및 필요성	소스코드 정적 분석은 자동화된 점검이 가능하지만, 오탐과 미탐이 발생하여 보안 전문가의 검증이 필요합니다. 머신러닝 알고리즘을 통해 취약한 코드와 안전한 코드를 학습시켜 AI 기반으로 탐지함으로써 이러한 문제를 해결할 수 있습니다. 이를 통해 오탐과 미탐을 줄일 수 있게 됩니다.
프로젝트 특징점	우리 프로젝트는 ARM 기반 명령어와 펌웨어의 취약점을 탐지하며, 머신러닝 알고리즘을 사용해 알려지지 않은 패턴도 파악할 수 있습니다. 소스코드를 LLVM IR로 변환해 모델에 학습시키고, 결과를 효과적으로 시각화한 PDF 보고서를 제공합니다.
주요 기능	이 프로젝트는 ARM 기반 펌웨어의 취약점을 탐지하기 위해 LLVM IR로 변환된 코드를 머신러닝 모델에 학습시킵니다. 전처리 과정에서 불필요한 정보를 제거하고, word2vec을 사용해 단어를 벡터화하며, LSTM 모델을 통해 데이터 학습을 진행합니다. 결과는 웹 인터페이스를 통해 업로드 및 점검 보고서 형태로 출력됩니다. 펌웨어 디컴파일에는 retdec 오픈소스를 사용하여 다양한 형식의 파일을 생성합니다.
기대효과 및 활용 분야	이 프로젝트는 자동으로 취약점을 탐지하고 보완 방법을 제안하여 코드 품질과 안정성을 개선합니다. 안전하지 않은 함수 호출 패턴을 탐지하고 개선 방법을 제시해 개발 및 유지 보수에서 시간과 비용을 절감합니다. AI 기반 도구의 한계를 넘어 다양한 보안 취약점 탐지 방법을 제공합니다.

(본문) 프로젝트 결과보고서

I. 프로젝트 개요

1. 프로젝트 소개

1)

[기획의도]

- ARM 아키텍처 기반 장비의 취약점 점검 지원

[프로젝트 내용 및 정의]

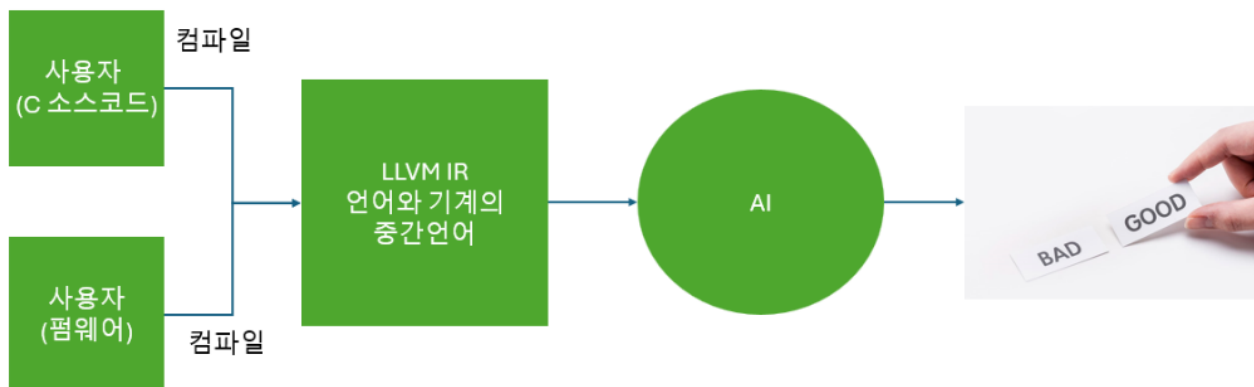
- 머신러닝 알고리즘을 이용해 취약 코드와 안전한 코드를 학습

* 학습데이터 : NIST에서 취약점이 있는 코드, 취약점이 없는 샘플 코드 제공

* <https://samate.nist.gov/SARD/test-suites/112>

- AI 기반으로 C언어 코드를 대상으로 취약성을 탐지

- 점검 결과를 PDF 또는 웹 형식으로 생성



2. 개발 배경 및 필요성

1)

C 파일

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

LLVM IR 파일

```
@.str = private unnamed_addr constant [15 x i8] c"Hello, World!\0A\00", align 1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
    %1 = alloca i32, align 4
    store i32 0, i32* %1, align 4
    %2 = call i32 @i8, ... @printf(i8 noundef getelementptr inbounds ([15 x i8], [15 x i8]* @.str, i64 0, i64 0))
    ret i32 0
}
```

* LLVM IR : 프로그래밍 언어와 기계어의 중간 레벨 언어이다.

기존의 정적 분석 도구는 코드의 복잡한 패턴을 분석하지 못해서 높은 오탐률을 가지고 있습니다. 이를 개선하기 위하여 2가지 방안을 채택하였습니다.

인공지능 :

1. AI 모델은 코드를 학습하고 패턴을 인식함으로써 기존 정적 분석보다 더 높은 정확도를 제공해 줌.

LLVM IR :

1. 컴파일러가 소스 코드를 최적화하고 코드 생성 단계로 전달하기 위해 사용하는 추상적인 명령어 집합
2. 코드의 실제 실행 동작을 보다 정확하게 분석할 수 있습니다.

3. 프로젝트 특 · 장점

1)

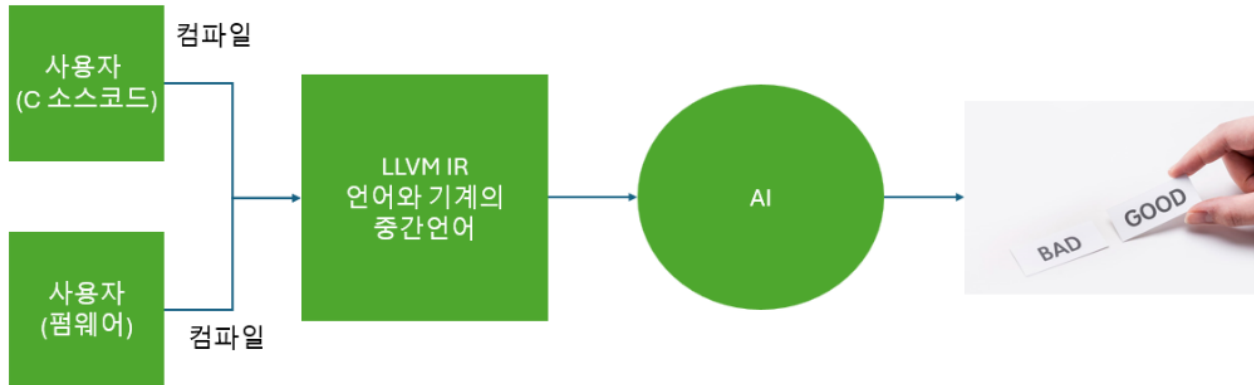
- ARM 아키텍처 기반 명령어의 취약점을 점검하여 ARM 기반 펌웨어의 취약점을 탐지 가능
- 머신러닝 알고리즘 사용으로 알려지지 않은 패턴 파악 가능
- 보고서 PDF 추출하여 효과적인 시각화에 도움을 줌.
- 기존 분석 도구들이 소스코드를 기반으로 분석하는 것과 달리 우리 프로젝트에서는 소스코드를 LLVM IR로 변환하여 모델에 학습시키는 형태로 진행

II. 프로젝트 내용

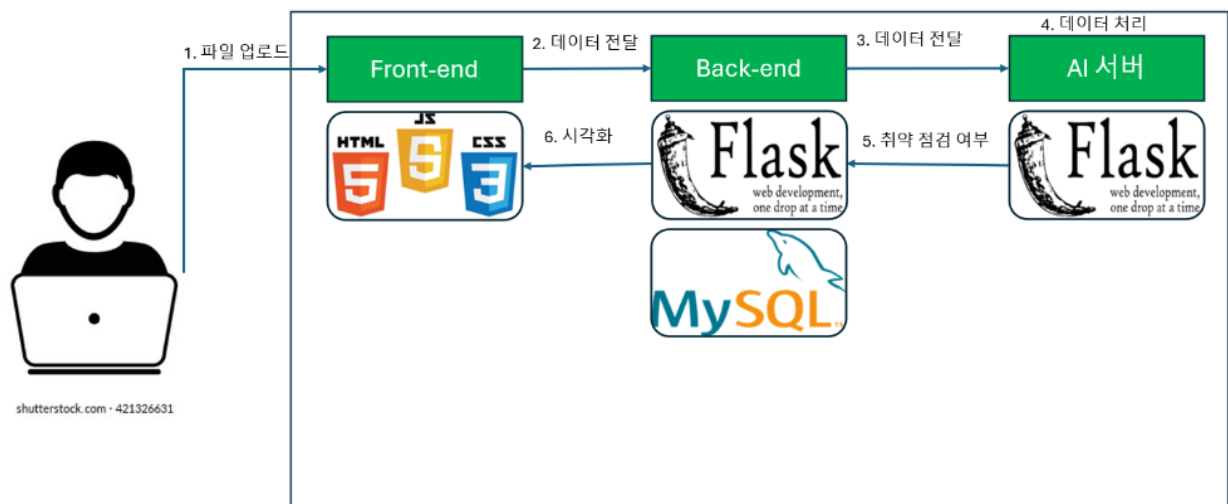
1. 프로젝트 구성도

1)

- 서비스 흐름도



- S/W 구성도

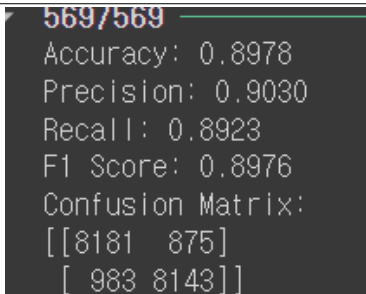


2. 프로젝트 기능

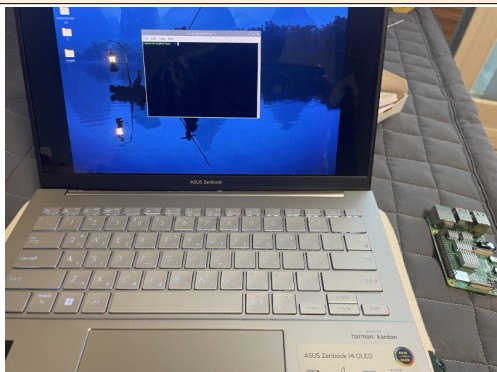
1) 전체 기능 목록

구분	기능	설명	현재진척도(%)
S/W	웹	취약점 분석 결과 시각화	80%
	인공지능 모델	취약점 식별	100%
H/W	라즈베리파이 (구매H/W)	ARM 아키텍처 펌웨어 제작(컴파일)	100%

2) S/W 주요 기능

기능	설명	프로젝트실물사진	
웹	- 파일 업로드 - 취약점 점검 결과 (유사CVE, CVSS스코어, 취약점 종류, 대응방법)		
인공지능	LSTM 학습	<pre> import tensorflow as tf model = Sequential() model.add(Embedding(input_dim=vocab_size, output_dim=embedding_dim, input_embeddings_initializer=ConstantEmbedderInitializer)) model.add(Bidirectional(LSTM(units=64, return_sequences=True, activation='tanh'))) model.add(Dropout(0.2)) model.add(Bidirectional(LSTM(units=64, return_sequences=True, activation='tanh'))) model.add(Dropout(0.2)) model.add(Bidirectional(LSTM(units=64, return_sequences=False, activation='tanh'))) model.add(Dropout(0.2)) model.add(Dense(1, activation='sigmoid')) optimizer = tf.keras.optimizers.Adam(learning_rate=0.001) model.compile(loss='binary_crossentropy', optimizer=optimizer, metrics=['accuracy']) </pre>	

3) H/W 주요 기능

기능/부품	설명	프로젝트실물사진
라즈베리파이4	ARM 아키텍처 컴파일	

특징	x86	ARM
아키텍처	CISC	RISC
명령어 집합	다양한 명령어	기본적인 명령어
용도	데스크탑, 서버, 고성능 컴퓨팅	모바일 장치, 임베디드 시스템
가격	비쌈, 고성능 하드웨어에 사용	저렴, 대량 생산에 적합

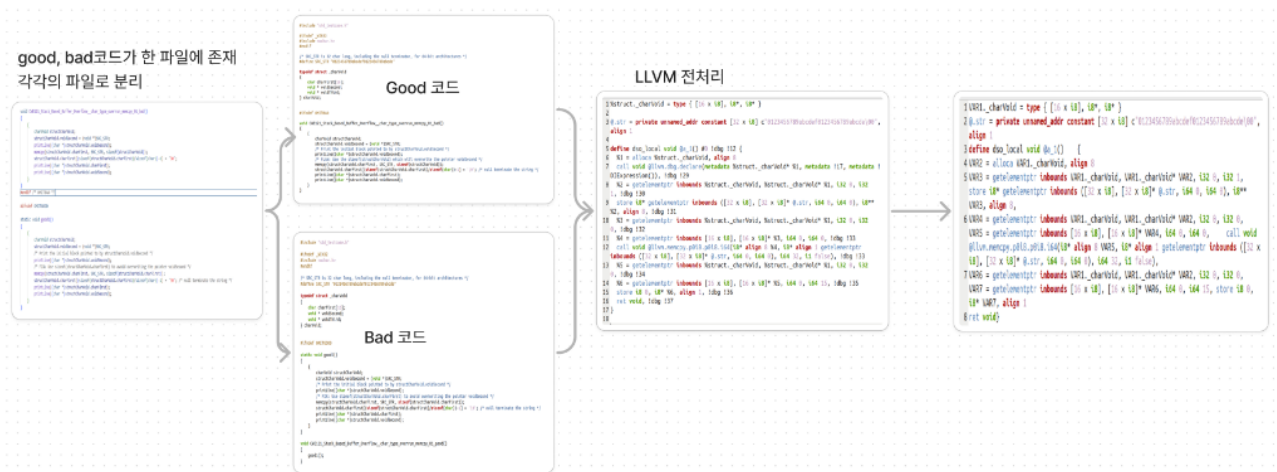
최근 IoT 장비인 공유기, 카메라등에 오래된 취약점에 존재하는 취약점을 찾기 위해 ARM기반의 데이터셋을 생성하기 위해 사용

3. 주요 적용 기술

- 인공지능 모델
- LSTM
- Word2vec

- 전처리

- Juliet 코드 good / bad 각각의 파일로 추출.
- C -> LLVM IR 변환
- LLVM IR 데이터 전처리
 - 메타, 플래그 데이터 제거
 - 전역 변수, 스택 포인터, 구조체 이름 변경
 - 불필요한 디버그 정보제거




Code BOom BOom

OWASP Top 10 CWE List OWASP Development Guidelines PDF

과기성적

선택된 과일이 없습니다.

Upload

© 2024 H1118. All Rights Reserved. Code BOomBOom. All rights reserved.


Code BOom BOom

OWASP Top 10 CWE List OWASP Development Guidelines PDF

```

1: /* TEMPLATE GENERATED TESTCASE FILE
2: Filename: CVE190_Integer_Overflow_char_fscanf_add_13.c
3: Label Definition File: CVE190_Integer_Overflow_label.xml
4: Template File source: sinks-13.template
5: */
6: /*
7: * @description
8: * CWE: 190 Integer Overflow
9: * BadSource: fscanf Read data from the console using fscanf()
10: * GoodSource: Set data to a small, non-zero number (two)
11: * Sinks: add
12: * GoodSink: (ensure there will not be an overflow before adding 1 to data)
13: * BadSink: Add 1 to data, which can cause an overflow
14: * Flow Variant: 13 Control flow: if(GLOBAL_CONST_FIVE==5) and if(GLOBAL_CONST_FIVE!=5)
15: */
16: /* */
17:
18: #include "std_testcase.h"
19:
20:
21: #ifndef OMITBAD
22:
23: void CWE190_Integer_Overflow_char_fscanf_add_13_bad()
24: {
25:     char data;
26:     data = '\0';
27:     if(GLOBAL_CONST_FIVE==5)
28:     {
29:         /* POTENTIAL FLAW: Use a value input from the console */
30:         fscanf(stdin, "%c", &data);
31:     }
32:     if(GLOBAL_CONST_FIVE==5)

```

새 취약점 결과

취약한 코드입니다.
Stack Overflow : 메모리의 스택 영역에서 할당된 메모리 크기를 초과하는 데이터를 쓰는 경우 발생할 수 있는 취약점입니다.

CVE 스코어: 9.8

관련 CVE
CVE-2024-0211 : A vulnerability identified as critical has been found in Tenda O6 1.0.0.7(2054). Affected is the function fromVirtualSet of the file /optom/softPortForward. The manipulation of the argument ipLocalPort/publicPort/loop leads to stack-based buffer overflow. It is possible to launch the attack remotely. The exploit has been disclosed to the public and may be used. NOTE: The vendor was contacted early about this disclosure but did not respond in any way.
CVE-2024-0210 : A vulnerability was found in Tenda O6 1.0.0.7(2054). It has been rated as critical. This issue affects the function fromSafeSetMchFilter of the file /optom/softModFilterList. The manipulation of the argument remark/type/time leads to stack-based buffer overflow. The attack may be initiated remotely. The exploit has been disclosed to the public and may be used. NOTE: The vendor was contacted early about this disclosure but did not respond in any way.
CVE-2024-0209 : A vulnerability was found in Tenda O6 1.0.0.7(2054). It has been declared as critical. This vulnerability affects the function fromMacFilterModify of the file /optom/operateMacFilter. The manipulation of the argument mac leads to stack-based buffer overflow. The attack can be initiated remotely. The exploit has been disclosed to the public and may be used. NOTE: The vendor was contacted early about this disclosure but did not respond in any way.

대응 방법

1. 입력 크기 제한 함수에 적용되는 입력의 크기를 엄격히 관리하여 비허용 데이터를 방지합니다.
2. 안전한 함수 사용: strcpy 또는 sprintf와 같은 안전한 문자열 처리 함수를 사용하여 비파괴를 보장합니다.

4. 프로젝트 개발 환경

구분		상세내용
S/W 개발환경	OS	Windows, Ubuntu
	개발환경(IDE)	VSCode
	개발도구	Colab, Retdec, Clang
	개발언어	Python
	기타사항	-
H/W 구성장비	디바이스	라즈베리파이4
	센서	-
	통신	-
	언어	-
	기타사항	-
프로젝트 관리환경	형상관리	Git
	의사소통관리	노션, Discord
	기타사항	-

5. 장비(기자재/재료) 활용

번호	품명	작품에서의 주요기능
1	라즈베리파이	- ARM 아키텍처 컴파일

6. 프로젝트 작동 동영상

- <https://www.youtube.com/watch?v=cUDbjL4hs7s>

7. 결과물 상세 이미지

복합 섹터
선택된 파일이 없습니다.
Upload

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

이해용 예제
업로드

업로드
업로드

8. 달성 성과

■ 논문게재 및 포스터발표	게재(발표)자명	논문(포스터)명	게재(발표)처	게재(발표)일자
	강인석	A 및 W를 활용한 정적 분석 기반 시큐어 코딩 점검 도구 연구	한국정보처리학회	2024. 11. 01.
□ 앱(APP) 등록	등록자명	앱(APP)명	등록처	등록일자
				2024. 00. 00.
□ 프로그램 등록	등록자명	프로그램명	등록처	등록일자
				2024. 00. 00.
□ 특허/실용신안 출원	출원자명	특허/실용신안명	출원번호	출원일자
				2024. 00. 00.
□ 기술이전	기술이전기업명	기술명	금액	이전일자
				2024. 00. 00.
■ 공모전	구분(교내/대외)	공모전명	수상여부(출품/수상)	상격
	대외	한이음 공모전	수상	입선
□ 실용화	#실용화한 내용에 대한 구체적 설명 기재			
□ 기타				

III. 프로젝트 수행 내용

1. 업무분장

번호	성명	역할	담당업무
1	서혁준	멘 토	프로젝트 방향성 제시
2		지도교수	# 지도교수가 없는 경우 공란으로 남김
3	강인석	팀 장	- 총괄 및 인공지능 모델 개발
4	조준형	팀 원2	- 데이터 전처리
5	박솔빈	팀 원3	- 웹 개발
6	윤건우	팀 원4	- 웹 개발
7	김보송	팀 원5	- 데이터 전처리

2. 프로젝트 수행일정

프로젝트 기간 (ICT멘토링 사이트 기준)		2024. 4. 1. ~ 2024. 10. 31.							
구분	추진내용	프로젝트 기간							
		3월	4월	5월	6월	7월	8월	9월	10월
계획	그라운드룰 협의								
분석	오픈소스 정적 분석 도구 분석								
설계	정적 분석 도구 설계								
	동적 분석 도구 설계								
	웹 페이지 포맷 설계								
개발	juliet 데이터셋 파싱 자동화 코드 구현								
	머신러닝 학습								
	펌웨어 역분석								
	웹 페이지 기능 구현								
테스트	통합 테스트								
종료 (성과등록)	공모전 참여								

3. 프로젝트 추진 과정에서의 문제점 및 해결방안

1) 프로젝트 관리 측면

- 멘티 5명이 모두 모일 수 있는 시간을 맞추는 게 정말 어려웠다.

각자 하는 일이 많았기 때문에 최소한의 진행사항의 공유가 반드시 필요했기 때문에 역할을 분배하여 같은 역할 인원이 회의를 하고 회의 내용을 전체 팀원에 공유하는 방법으로 해결하였다.

2) 프로젝트 개발 측면

- 바이너리를 디컴파일 하는 과정에서 펌웨어 파일의 경우(.bin) 확장자를 가지고 있기 때문에 이 파일을 그대로 LLVM IR로 변환할 수 있을 것이라고 생각하였음. 하지만 펌웨어 파일에는 squashfs라는 파일시스템이 존재하기 때문에 mount 한 파일시스템에서 find 명령어를 사용하여 ELF 파일을 찾아내는 방식으로 한다는 것을 알게 되었다.

=> 멘토님께서 squashfs를 mount 해보면 좋을 것이라고 말씀을 해주셔서 문제를 해결할 수 있었음.

4. 프로젝트를 통해 배우거나 느낀 점

- 프로젝트를 진행하면서 Recdec이라는 강력한 디컴파일 도구를 발견해 큰 도움이 되었고, 디컴파일 도구에 대한 학습이 이루어졌습니다. 성공적인 프로젝트 진행을 위해 상세한 계획 수립의 중요성을 깨달았습니다. 긴 프로젝트 기간 동안 여유를 가지기보다 완성도를 높일 방법을 모색하고, 역할을 원활히 분배하는 것이 중요하다는 점을 인식했습니다. 초기 작업이 적었지만, 마무리를 위해 더 많은 시간을 투자하고, 남은 기간 동안 효과적으로 분배하며 완성도 높은 프로젝트를 목표로 해야겠습니다.

IV. 기대효과 및 활용분야

1. 프로젝트의 기대효과

자동 취약점 검출 및 보안 - 패턴을 자동으로 탐지하고 보완 방법 제안
코드 품질 개선 및 안정성 강화 - 안전하지 않은 함수 호출 패턴을 탐지하여 개선
효율적인 개발 및 유지 보수 - 해당 문제를 해결하는 방법을 제시함으로써 개발 및 유지 보수에서의 시간과 비용을 절감할 수 있다.
한정된 시큐어 코딩 도구의 다양성 - AI 기반의 도구는 제한적이다 대부분은 정적 분석, 패턴 기반으로 한 보안 취약점 탐지 방법을 사용한다.

2. 프로젝트의 활용분야

- 하드웨어로 송수신되는 패킷을 자동으로 캡처하여 머신러닝 기반으로 분석하는 동적 분석 도구 개발,
- 소스코드의 취약성 유무를 머신러닝 기반으로 분석하는 정적 분석 도구 개발
- github를 자동으로 스캔하여 취약한 코드를 작성한 저자에게 알림을 주는 안심 서비스 제공

V. 참고자료

1. 참고 및 인용자료

- Static Code Analysis of C++ in LLVM

<https://www.diva-portal.org/smash/get/diva2:930827/FULLTEXT01.pdf>

- LLVM Intermediate Representation for Code Weakness Identification

<https://insights.sei.cmu.edu/library/llvm-intermediate-representation-for-code-weakness-identification/>

- Bi-Directional Transformers vs. word2vec: Discovering Vulnerabilities in Lifted Compiled Code

<https://arxiv.org/abs/2405.20611>

별첨1

2024년 ICT멘토링 프로젝트 성과 증빙자료

대표 정보

연구 유형 ICT멘토링(한이음) 학부생논문

발표 유형 구두(대면)

논문 번호 KIPS_C2024B0112 (상태: 최종본 제출 완료)

제목 AI 및 LLVM을 활용한 정적 분석 기반 시큐어 코딩 점검 도구 연구

발표분야 C6. 정보보안

초록 이 연구는 LLVM IR을 활용한 보안 취약점 탐지의 새로운 접근 방식을 제시합니다. Juliet Test Suite의 CWE-121 Stack-based Buffer Overflow 데이터를 사용하여 Word2Vec으로 코드를 벡터화하고 LSTM 모델로 학습했습니다. 모델 성능은 정확도 90%, 정밀도 87%, 재현율 93%, F1 스코어 90%로 평가되었습니다. 향후 다양한 보안 취약점을 다룰 수 있는 다중 분류 모델로 확장 가능성을 제안합니다.

주제어 (Keywords)

Buffer Overflow C LLVM IR LSTM Word2Vec 시큐어코딩

투고자

강인석, 창원대학교, 컴퓨터공학과
김보송, 성신여자대학교, 융합보안공학과
박슬빈, 홍익대학교, 자율전공
윤건우, 가천대학교, 스마트보안학과
조준형, 단국대학교, 소프트웨어학과
서혁준, 엘지씨엔에스, RED팀

최종 게재 여부 결정

게재 여부 결정 ●게재 수락 (2024-10-10)

프로젝트번호	프로젝트명	트랙	프로젝트수행기간	참여인원	상태	신청
19804	AI 기반의 소스 및 바이너리 시큐어 코딩 점검 도구	한이음	2024-04-01 ~ 2024-10-31	6명	입선	내용 보기